Docker Setup — Developer Onboarding Guide

This document covers everything you need to understand, run, and modify the Docker setup for Pollinator Habitat. It explains each compose file, each Dockerfile, the Nginx reverse proxy, and how all the pieces fit together.

Table of Contents

---

1. Prerequisites

- **Docker Desktop** (or Docker Engine + Docker Compose v2)
- **Git** — to clone the repo
- No Node.js or npm required on the host machine — everything runs inside containers

Verify Docker is working:

```
docker --version
docker compose version
```

---

2. Repository Layout — Docker Files at a Glance

```
Pollinator-Habitat/
|
```

```
├── docker-compose.yml          ← MySQL PRODUCTION stack
├── docker-compose.dev.yml      ← MySQL DEVELOPMENT stack (hot reload)
├── docker-compose.prod.pg.yml  ← PostgreSQL PRODUCTION stack
├── docker-compose.dev.pg.yml   ← PostgreSQL DEVELOPMENT stack (hot reload)
│
├── .env                        ← MySQL production credentials (gitignored)
├── .env.pg.example             ← PostgreSQL credentials template
│
├── nginx/
│   └── nginx.conf              ← Reverse proxy config (production stacks only)
│
├── mysql-init/                 ← Optional SQL scripts run at MySQL first-start
│
├── backend/
│   ├── Dockerfile              ← Backend production image (MySQL flavour)
│   ├── Dockerfile.pg           ← Backend production image (PostgreSQL flavour)
│   └── dockerfile.dev          ← Backend dev image (shared by both dev stacks)
│
├── frontend/
│   ├── Dockerfile              ← Frontend production image (shared by both prod
stacks)
│   └── dockerfile.dev          ← Frontend dev image
│
└── portal/
    ├── Dockerfile              ← Portal production image (shared by both prod stacks)
    └── dockerfile.dev          ← Portal dev image
```

---

3. The Four Compose Files

File	Database	Purpose	Nginx?	Hot reload?
`docker-compose.yml`	MySQL 8.4	Production	Yes (ports 80, 3001)	No
`docker-compose.dev.yml`	MySQL 8.4	Local development	No	Yes
`docker-compose.prod.pg.yml`	PostgreSQL 17	Production	Yes (ports 80, 3001)	No
`docker-compose.dev.pg.yml`	PostgreSQL 17	Local development	No	Yes

Which one should I use?

- Day-to-day development → `docker-compose.dev.pg.yml` (PostgreSQL, recommended) or `docker-compose.dev.yml` (MySQL, simpler credentials)
- Deploying to a server → `docker-compose.prod.pg.yml` (PostgreSQL, no licensing issues)
- See MYSQL_LICENSE_ISSUE.md for why PostgreSQL is preferred for any non-personal deployment

---

4. Choosing a Stack

First time setup

Development (PostgreSQL):

```
cd Pollinator-Habitat/
docker compose -f docker-compose.dev.pg.yml up
```

No `.env` file needed — credentials are hardcoded in the compose file.

Development (MySQL):

```
cd Pollinator-Habitat/
# JWT_SECRET must be set
JWT_SECRET=any_local_secret docker compose -f docker-compose.dev.yml up
# or add it to a .env file:
echo "JWT_SECRET=any_local_secret" > .env
docker compose -f docker-compose.dev.yml up
```

Production (PostgreSQL):

```
cd Pollinator-Habitat/
cp .env.pg.example .env.pg
# Edit .env.pg — change the JWT_SECRET before deploying
docker compose -f docker-compose.prod.pg.yml up -d
```

---

5. Development Stacks — How They Work

Development stacks prioritize **fast iteration**:

- Source files are mounted into the containers as volumes — you edit on the host, the container sees the change immediately
- The backend uses `ts-node --watch` (`npm run dev`) — restarts automatically on TypeScript changes
- The frontend and portal use Next.js dev server — hot module replacement (HMR)
- No production build step required

MySQL Dev Stack (`docker-compose.dev.yml`)

Services: `backend` , `frontend` , `portal` , `mysql` , `migrate`

Credentials (hardcoded, no `.env` needed):

```
MySQL root password: rootpw
Database:           pollinator
User:               pollinator
Password:           pollinatorpw
Shadow DB:          pollinator_shadow
```

Ports exposed to host:

Service	Host port	Container port
backend	`127.0.0.1:4000`	`4000`
frontend	`127.0.0.1:80`	`3000`
portal	`127.0.0.1:3001`	`3000`
mysql	`127.0.0.1:3306`	`3306`

All ports are bound to `127.0.0.1` (loopback only) — not accessible from other machines on the network.

Portal API URL in dev: `NEXT_PUBLIC_API_URL=http://localhost:4000` — the portal calls the backend directly on port 4000 (no Nginx in dev).

Frontend API URL in dev: `NEXT_PUBLIC_API_URL=""` — empty string, so API calls are relative (`/api/...`). The frontend's Next.js dev server is on port 80, and there's no proxy, so this works because the Next.js app and the browser are on the same origin. **Actually**, the frontend uses an empty string meaning it calls `/api/...` relative to its own origin (port 80) — which works in dev because the frontend container itself is on port 80. However, the backend is on 4000. This means frontend `/api/` calls must be proxied — or the Next.js config rewrites them. In practice, check whether a `rewrites()` config or Nginx is handling this in your dev setup.

Volume mounts (MySQL dev): The backend mounts the entire repo root (`. → /app`) with two exclusions to prevent overwriting installed packages:

```
volumes:
  - .:/app
  - /app/node_modules          ← uses container's node_modules, not host's
  - /app/backend/node_modules  ← uses container's backend/node_modules
```

The frontend and portal follow the same pattern.

PostgreSQL Dev Stack (`docker-compose.dev.pg.yml`)

Services: `backend` , `frontend` , `portal` , `postgres` , `migrate`

Credentials (hardcoded):

```
Database:  pollinator
User:      pollinator
Password:  pollinatorpw
Port:      5432
```

`DATABASE_URL=postgresql://pollinator:pollinatorpw@postgres:5432/pollinator`

Ports exposed to host:

Service	Host port	Container port
backend	`127.0.0.1:4000`	`4000`
frontend	`127.0.0.1:80`	`3000`

portal	`127.0.0.1:3001`	`3000`
postgres	`127.0.0.1:5432`	`5432`

Volume mounts (PG dev): More targeted than MySQL dev — mounts specific subdirectories instead of the entire repo:

```
# backend
volumes:
  - ./backend/src:/app/backend/src
  - ./backend/prisma:/app/backend/prisma
  - ./shared:/app/shared

# frontend
volumes:
  - ./frontend/src:/app/frontend/src
  - ./frontend/public:/app/frontend/public
  - ./tsconfig.base.json:/app/tsconfig.base.json:ro
  - ./shared:/app/shared

# portal
volumes:
  - ./portal/src:/app/portal/src
  - ./portal/public:/app/portal/public
```

This is more precise — only source and public assets are hot-reloaded; package.json, configs, and node_modules are untouched until you rebuild the image.

Backend startup command (PG dev):

```
npx prisma generate --config prisma.config.pg.ts && npm run dev
```

Note: uses `prisma.config.pg.ts` (the PostgreSQL Prisma config) instead of the default.

Migrate command (PG dev):

```
npx prisma migrate deploy --config prisma.config.pg.ts \
  && npx prisma generate --config prisma.config.pg.ts \
  && npx tsx prisma/seed.pg.js
```

Uses `seed.pg.js` (the PostgreSQL-compatible seed script) instead of `seed.js`.

Hot Reload and Volume Mounts

Hot reload works via file system polling because the host OS (macOS) and the Linux container use different file systems — inotify events don't cross the Docker volume boundary reliably.

How polling is enabled:

```
environment:
  CHOKIDAR_USEPOLLING: "true"    # backend (ts-node/chokidar)
```

```
    WATCHPACK_POLLING: "true"      # frontend and portal (Next.js / webpack)
```

These environment variables tell the respective file watchers to poll instead of listening for native FS events. Polling adds slight latency (~1s) but is reliable across all host OSes.

---

6. Production Stacks — How They Work

Production stacks prioritize **security and minimal image size**:

- All images are built from multi-stage Dockerfiles into distroless runtime images
- No source files in the final image — only compiled output
- No `devDependencies` in the final image — pruned before the runtime stage
- No exposed ports on individual services — everything goes through Nginx

Service Startup Order

The `depends_on` conditions enforce a strict startup sequence:

```
postgres/mysql
    │ (service_healthy — waits for DB to accept connections)
    ▼
migrate
    │ (service_completed_successfully — waits for migrations AND seed to finish)
    ▼
backend
    │ (service_started)
    ▼
frontend, portal
    │ (service_healthy — waits for Next.js to respond on port 3000)
    ▼
nginx
```

What `service_healthy` means: Docker runs the service's `healthcheck` command on a loop. `service_healthy` means the check has passed at least once. The backend does not start until the database health check passes — preventing Prisma connection errors on startup.

What `service_completed_successfully` means: The `migrate` container must exit with code 0 before the backend starts. This guarantees the schema is migrated and seed data is loaded before the backend accepts any requests.

Nginx Reverse Proxy

File: `nginx/nginx.conf`

Nginx is the only service with externally exposed ports in production:

- Port `80` → player frontend
- Port `3001` → staff portal

Both servers proxy `/api/` and `/api` (exact match) to the backend. All other paths go to their respective Next.js server.

Port 80 server block:

```
/api    → proxy to http://backend:4000
/api/   → proxy to http://backend:4000
/       → proxy to http://frontend:3000
```

Port 3001 server block:

```
/api    → proxy to http://backend:4000
/api/   → proxy to http://backend:4000
/       → proxy to http://portal:3000
```

Why two separate `/api` location blocks?

Nginx has a quirk: `location /api/` matches `/api/start-game` but **not** the exact path `/api` (no trailing slash). A separate `location = /api` (exact match) handles the bare `/api` path. Without it, `POST /api` (the session validation endpoint) would not be proxied.

Why `set $backend_upstream` and `proxy_pass $backend_upstream$request_uri`?

When `proxy_pass` uses a literal URL (e.g. `proxy_pass http://backend:4000`), Nginx resolves the hostname once at startup and caches it. If the `backend` container restarts and gets a new IP, Nginx uses the stale address and requests fail.

By assigning the upstream to a variable, Nginx is forced to re-resolve the hostname on every request using Docker's internal DNS (`resolver 127.0.0.11 valid=10s`). This makes Nginx resilient to container restarts.

Why `NEXT_PUBLIC_API_URL=""`?

In production, the frontend and portal both have `NEXT_PUBLIC_API_URL=""`. Their fetch calls go to `/api/...` (relative URL). Since Nginx is the entry point, a request from the browser to `http://host/api/start-game` goes to Nginx (port 80) first, which proxies it to the backend. The apps never need to know the backend's direct address.

Health Checks

Database health check (MySQL):

```
test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-uroot", "-
p${MYSQL_ROOT_PASSWORD}"]
interval: 5s
timeout: 3s
retries: 30
```

Retries 30 times over up to 150 seconds. MySQL takes time to initialise on first boot.

Database health check (PostgreSQL):

```
test: ["CMD-SHELL", "pg_isready -U pollinator -d pollinator"]
interval: 5s
timeout: 3s
retries: 30
```

Frontend / Portal health check:

```
test: ["CMD", "/nodejs/bin/node", "-e",
  "require('http').get('http://localhost:3000/', r => process.exit(r.statusCode <
400 ? 0 : 1)).on('error', () => process.exit(1))"]
interval: 10s
timeout: 5s
retries: 5
start_period: 30s
```

Uses Node.js (already in the distroless image) to make an HTTP GET to itself. The `start_period: 30s` gives Next.js time to boot before the first health check runs.

---

7. The Dockerfiles

Backend Production (`Dockerfile` / `Dockerfile.pg`)

Both files are nearly identical — 4 stages:

Stage 1: `deps` — installs npm dependencies

```
FROM node:24.14.0-slim AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY backend/package.json ./backend/package.json
RUN npm ci
```

Uses `npm ci` (not `npm install`) — installs exactly what is in `package-lock.json` , fails if there is a mismatch. Fast and deterministic.

Stage 2: `build` — compiles TypeScript and generates Prisma client

```
FROM node:24.14.0-slim AS build
# ... copies source, shared, tsconfig
WORKDIR /app/backend
RUN npx prisma generate [--config prisma.config.pg.ts]  ← .pg version only
RUN npm run build
```

`prisma generate` must run before `tsc` because the compiled code imports from the `generated/` folder. The MySQL `Dockerfile` passes dummy database credentials at generate time (Prisma needs a URL for validation even if the DB isn't running); the PG `Dockerfile.pg` doesn't need them because `prisma.config.pg.ts` already points at the correct schema.

Stage 3: `prune` — removes devDependencies

```
FROM node:24.14.0-slim AS prune
# copies node_modules from build stage
RUN npm prune --omit=dev
```

The `prune` stage is separate so that `devDependencies` (TypeScript compiler, test tools, etc.) are stripped before the runtime image is assembled.

Stage 4: `runtime` — final distroless image

```
FROM gcr.io/distroless/nodejs24-debian12:nonroot AS runtime
WORKDIR /app
ENV NODE_ENV=production

COPY --from=prune  --chown=65532:65532 /app/node_modules ./node_modules
COPY --from=build  --chown=65532:65532 /app/backend/dist ./backend/dist
COPY --from=build  --chown=65532:65532 /app/backend/generated ./backend/generated
# ... package.json files

EXPOSE 4000
CMD ["backend/dist/backend/src/index.js"]
```

Distroless means the image has no shell, no package manager, no OS utilities — only the Node.js runtime and the app. Benefits:

- Smallest possible attack surface (no shell to exploit)
- Smaller image size
- Runs as user `65532` (nonroot) — never runs as root

The `CMD` points directly at the compiled entry point JS file (not `node src/index.js` — the compiled output is in `dist/`).

Frontend & Portal Production (`frontend/Dockerfile` / `portal/Dockerfile`)

Same 4-stage pattern as the backend. The only differences:

- Stage 2 runs `npm run build` (Next.js build — generates the `.next/` folder)
- Stage 4 copies `.next/`, `public/`, and `next.config.*` instead of `dist/` and `generated/`
- `CMD` is `["node_modules/next/dist/bin/next", "start", "frontend"]` (or `portal`) — runs the Next.js production server

The portal's `Dockerfile` copies `next.config.ts` (TypeScript config); the frontend's copies `next.config.mjs` (ES module config) — different extensions because they were written at different times.

Dev Dockerfile — Backend (`backend/dockerfile.dev`)

```
FROM node:22-alpine
RUN apk add --no-cache openssl   ← required by Prisma
WORKDIR /app
COPY package*.json ./
COPY backend/package*.json backend/
COPY frontend/package*.json frontend/
COPY portal/package*.json portal/
RUN mkdir -p shared && echo '...' > shared/package.json  ← stub for npm workspaces
RUN npm install
WORKDIR /app/backend
COPY backend .
```

```
ENV CHOKIDAR_USEPOLLING=true
EXPOSE 4000
CMD ["npm", "run", "dev"]
```

Key differences from production:

- Uses `node:22-alpine` (not `24-slim`) — lighter base
- `npm install` (not `npm ci`) — installs all deps including `devDependencies`
- `CHOKIDAR_USEPOLLING=true` — enables file polling for hot reload
- `CMD ["npm", "run", "dev"]` — runs `ts-node --watch`, not the compiled output
- Prisma binary requires `openssl` which is not in alpine by default — installed via `apk`

The compose file **overrides** the `CMD` at runtime:

```
command: sh -c "npx prisma generate && npm run dev"
```

This runs `prisma generate` first (to ensure the client is up to date before starting the server), then starts `ts-node --watch`.

Dev Dockerfiles — Frontend & Portal (`frontend/dockerfile.dev`, `portal/dockerfile.dev`)

```
FROM node:22-alpine
WORKDIR /app/frontend   # (or /app/portal)
COPY frontend/package*.json ./
RUN npm install
COPY frontend .
ENV CHOKIDAR_USEPOLLING=true
ENV WATCHPACK_POLLING=true
EXPOSE 3000
CMD ["npm", "run", "dev"]
```

Minimal — alpine Node, install deps, copy source, start Next.js dev server. Source files are overwritten at container startup by the volume mount in the compose file, so the `COPY frontend .` step is effectively a fallback for the first install.

---

8. Environment Variables

Development (MySQL) — no `.env` needed

MySQL dev credentials are hardcoded in `docker-compose.dev.yml`. Only `JWT_SECRET` is required from the environment:

```
JWT_SECRET=any_local_dev_secret docker compose -f docker-compose.dev.yml up
```

Or put it in a `.env` file in the `Pollinator-Habitat/` directory:

```
JWT_SECRET=any_local_dev_secret
```

Development (PostgreSQL) — no `.env` needed

All credentials, including `DATABASE_URL`, are hardcoded in `docker-compose.dev.pg.yml`. `JWT_SECRET` is still read from the environment (the file has `JWT_SECRET: ${JWT_SECRET}`).

Production (MySQL) — requires `.env`

The production `docker-compose.yml` reads from `.env` (Docker Compose auto-loads `.env` from the same directory):

Variable	Purpose
`MYSQL_ROOT_PASSWORD`	MySQL root password
`MYSQL_DATABASE`	Database name (e.g. `pollinator`)
`MYSQL_USER`	App database user
`MYSQL_PASSWORD`	App database password
`DATABASE_URL`	Prisma connection string
`SHADOW_DATABASE_URL`	Prisma shadow DB for migrations
`JWT_SECRET`	Signs and verifies JWT tokens

Production (PostgreSQL) — requires `.env.pg`

The production `docker-compose.prod.pg.yml` reads from `.env.pg.example` (the compose file points at `.env.pg.example` directly via `env_file`). Copy and rename before deploying:

```
cp .env.pg.example .env.pg
```

Then edit `.env.pg` and change the compose file's `env_file` to `.env.pg`. The example file contains:

```
POSTGRES_DB=pollinator
POSTGRES_USER=pollinator
POSTGRES_PASSWORD=pollinatorpw  ← change this
DATABASE_URL=postgresql://pollinator:pollinatorpw@postgres:5432/pollinator  ← match
above
JWT_SECRET=...  ← MUST change this before deploying
```

Critical: Never deploy with the example `JWT_SECRET`. It is a public placeholder. Generate a new one:

```
openssl rand -hex 32
```

---

9. The Migrate Service

The `migrate` service is a one-shot container (`restart: "no"`) that runs database migrations and seeds before the backend starts.

What it runs:

MySQL stacks:

```
npx prisma migrate deploy && npx prisma generate && npx tsx prisma/seed.js
```

PostgreSQL dev stack:

```
npx prisma migrate deploy --config prisma.config.pg.ts \
  && npx prisma generate --config prisma.config.pg.ts \
  && npx tsx prisma/seed.pg.js
```

PostgreSQL prod stack:

```
npx prisma migrate deploy --config prisma.config.pg.ts && npx tsx prisma/seed.pg.js
```

`prisma migrate deploy` — applies all pending migrations to the database. Unlike `migrate dev`, it does not create new migrations or prompt for input — safe to run in CI/CD.

`prisma generate` — regenerates the Prisma client from the schema. In dev stacks this is included here; in prod stacks it runs during the `build` stage of the Dockerfile instead.

`prisma/seed.js` / `seed.pg.js` — populates the database with initial data: sessions, routes, route nodes, fact nodes, and pollinators. Pre-seeded session IDs: `18429`, `57243`, `90618`, `34790`, `62851`.

Why a separate container for migrations?

Running `migrate deploy` and `seed` inside the backend container at startup would mean: if the migration fails, the backend crashes and Docker restarts it, triggering the migration again, possibly in a bad state. By isolating migration into its own container with `restart: "no"`, a failure is surfaced cleanly and Docker does not retry.

In dev stacks, the `migrate` container also uses `backend/dockerfile.dev` as its image — it has all the tools needed (`prisma`, `tsx`). In production stacks, it uses `target: build` of the backend Dockerfile — the `build` stage has everything needed to run migrations.

---

10. The Network

All services share a single Docker bridge network: `pollinator_net`.

```
networks:
  pollinator_net:
    driver: bridge
```

This means:

- Containers can reach each other by **service name** as hostname (e.g. `mysql:3306`, `backend:4000`, `frontend:3000`)
- Containers are isolated from the host and from other Docker networks
- In production, only Nginx has ports exposed to the host — the backend, frontend, portal, and database are not directly reachable from outside Docker

Why service names work as hostnames: Docker's embedded DNS server automatically creates DNS entries for each service on the bridge network. When the backend connects to `mysql:3306`, Docker resolves `mysql` to the MySQL container's internal IP.

---

11. Volumes (Persistent Data)

Production stacks create named volumes for the database data:

```
volumes:
  pollinator-mysql-data:   # MySQL
  pollinator-postgres-data: # PostgreSQL
```

Named volumes persist across `docker compose down` and container restarts. Data is not lost when you restart the stack.

To completely reset the database (drop all data):

```
docker compose -f docker-compose.prod.pg.yml down -v
# -v removes named volumes
docker compose -f docker-compose.prod.pg.yml up
# migrate will re-run seed on fresh DB
```

Development stacks also use named volumes (`mysql_data` , `postgres_data`) for the same reason — your dev data survives container restarts.

---

12. Common Tasks

Start the dev stack

```
cd Pollinator-Habitat/
# PostgreSQL:
docker compose -f docker-compose.dev.pg.yml up

# MySQL:
JWT_SECRET=devSecret docker compose -f docker-compose.dev.yml up
```

Start in detached mode (background)

```
docker compose -f docker-compose.dev.pg.yml up -d
```

View logs for a specific service

```
docker compose -f docker-compose.dev.pg.yml logs -f backend
docker compose -f docker-compose.dev.pg.yml logs -f migrate
```

Stop everything

```
docker compose -f docker-compose.dev.pg.yml down
```

Rebuild an image after changing a Dockerfile or package.json

```
docker compose -f docker-compose.dev.pg.yml up --build backend
```

Re-run migrations manually

```
docker compose -f docker-compose.dev.pg.yml run --rm migrate
```

Connect to PostgreSQL directly (dev)

```
docker exec -it pollinator-postgres-dev psql -U pollinator -d pollinator
```

Connect to MySQL directly (dev)

```
docker exec -it pollinator-mysql mysql -u pollinator -ppollinatorpw pollinator
```

Wipe and reseed the database

```
docker compose -f docker-compose.dev.pg.yml down -v
docker compose -f docker-compose.dev.pg.yml up
```

---

13. Troubleshooting

`migrate` exits with error on first run

Usually means the database wasn't fully ready. The `service_healthy` condition should prevent this, but on slow machines the healthcheck might pass before MySQL/Postgres is truly ready to accept connections.

Fix: Re-run the migrate container:

```
docker compose -f docker-compose.dev.pg.yml run --rm migrate
```

Or bring the stack down and back up:

```
docker compose -f docker-compose.dev.pg.yml down && docker compose -f docker-compose.dev.pg.yml up
```

Backend crashes with `JWT_SECRET is not set`

`JWT_SECRET` is required at startup. Check that it is set in your environment or `.env` file.

Hot reload not working

1. Check that `CHOKIDAR_USEPOLLING=true` is set for the backend and `WATCHPACK_POLLING=true` for Next.js services — these are in the compose files by default.
2. Check that your editor saves files to the mounted directory on the host, not a temp location.
3. Try restarting the container: `docker compose restart backend`.

Port conflict on host startup

If `docker compose up` reports a port already in use:

```
# Find what's using port 80:
lsof -i :80
# or on Linux:
ss -tulpn | grep :80
```

Either stop the conflicting process or change the host-side port mapping in the compose file (e.g. `"127.0.0.1:8080:3000"` instead of `"127.0.0.1:80:3000"`).

`prisma generate` fails in the build stage

The MySQL `Dockerfile` passes dummy credentials for `prisma generate`:

```
RUN DATABASE_URL="mysql://user:pass@localhost:3306/db" SHADOW_DATABASE_URL=... npx prisma generate
```

If you're getting errors here, check that the schema file exists and is valid. Prisma validates the schema even with dummy URLs.

Image won't rebuild with new dependencies

Docker caches `npm ci` layers. If you add a dependency to `package.json`, you must force a rebuild:

```
docker compose -f docker-compose.dev.pg.yml up --build
```

`connection refused` to backend from portal in dev

In dev, `NEXT_PUBLIC_API_URL=http://localhost:4000` for the portal. This is resolved from the **host machine's** perspective (the browser), not from inside the container. If you're accessing the portal from a different machine, change this to the host's IP/hostname.

---

14. Dockerfile Reference

Each application has a multi-stage production Dockerfile. All use the same four-stage pattern: `deps → build → prune → runtime`.

Stage pattern

Stage	Base image	Purpose

`deps`	`node:24.14.0-slim`	Install all npm dependencies (including devDeps)
`build`	`node:24.14.0-slim`	Copy source, run TypeScript compiler / Next.js build
`prune`	`node:24.14.0-slim`	Remove devDependencies (`npm prune --omit=dev`)
`runtime`	`gcr.io/distroless/nodejs24-debian12:nonroot`	Minimal runtime — no shell, no package manager

Why distroless: The runtime image (`gcr.io/distroless/nodejs24-debian12:nonroot`) contains only the Node.js runtime and the OS libraries it needs. No shell, no `apt` , no `curl` . This minimizes the attack surface and image size. The `:nonroot` tag runs as user `65532` (not root).

Why multi-stage: The `deps` and `build` stages use a full Node.js image to compile and install. Only the compiled output and pruned `node_modules` are copied into the final `runtime` stage, so build tools are never in the production image.

`backend/Dockerfile` (MySQL/MariaDB)

```
deps   → install all deps
build  → copy source, generate Prisma client (dummy credentials), run tsc
prune  → strip devDeps
runtime → copy dist/ + generated/ + node_modules
          EXPOSE 4000
          CMD ["backend/dist/backend/src/index.js"]
```

`prisma generate` in the build stage: Prisma requires `DATABASE_URL` to generate the client. A dummy MySQL URL is passed at build time — Prisma only needs it to validate schema syntax, not to connect.

`backend/Dockerfile.pg` (PostgreSQL)

Identical to `Dockerfile` except:

- `prisma generate` uses `--config prisma.config.pg.ts` to target the PostgreSQL schema
- No dummy `SHADOW_DATABASE_URL` needed (PostgreSQL doesn't use shadow DB)

`frontend/Dockerfile`

```
deps   → install all deps
build  → copy source + shared/, run next build
prune  → strip devDeps
runtime → copy next.config.mjs + public/ + .next/
           EXPOSE 3000
           CMD ["node_modules/next/dist/bin/next", "start", "frontend"]
```

The Next.js `start` command is passed `frontend` as the working directory argument because the monorepo root is `/app` but the Next.js project is in `/app/frontend` .

`portal/Dockerfile`

Identical to `frontend/Dockerfile` pattern. Builds from `portal/` instead of `frontend/`. Copies `portal/public/` in addition to the compiled `.next/` output.

Dev Dockerfiles (`backend/dockerfile.dev`, `frontend/dockerfile.dev`, `portal/dockerfile.dev`)

The dev Dockerfiles are single-stage and run `npm run dev` (with hot reload). They mount the source directories as Docker volumes so code changes on the host are immediately reflected in the container without rebuilding.

---

15. Nginx Configuration Reference

File: `nginx/nginx.conf`

The Nginx configuration creates two virtual servers:

Port 80 — Player Game

```
server {
  listen 80;

  location = /api { proxy_pass http://backend:4000; }
  location /api/  { proxy_pass http://backend:4000; }
  location /      { proxy_pass http://frontend:3000; }
}
```

- Exact match `location = /api` handles `POST /api` (the JWT handshake, which has no trailing slash).
- Prefix match `location /api/` handles all other API paths (`/api/start-game`, `/api/complete-route`, etc.).
- Both are needed because a `POST /api` without a trailing slash would be 301-redirected to `/api/` by most proxy configs, which changes the method to `GET`. Nginx avoids this with the exact match.
- Everything else (`/`, `/home`, `/route`, `/pollinator-collection`, etc.) proxies to the Next.js frontend.

Port 3001 — Staff Portal

Identical routing structure, but the catch-all proxies to `portal:3000` instead of `frontend:3000`.

DNS re-resolution trick

```
set $backend_upstream http://backend:4000;
proxy_pass $backend_upstream$request_uri;
```

By storing the upstream in a variable, Nginx forces re-resolution via Docker's internal DNS (`resolver 127.0.0.11 valid=10s`). Without this, Nginx resolves the hostname once at startup and caches it — if the

backend container restarts and gets a new IP, Nginx would continue sending to the old IP until it itself restarted. Using a variable bypasses the static cache.

Proxy headers

Each location sets:

```
proxy_set_header Host              $host;
proxy_set_header X-Real-IP         $remote_addr;
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

These pass the original client IP and protocol to the upstream services. The backend uses `X-Real-IP` in the rate limiter to track requests per client IP (not per proxy IP).

Not tracked by git — update when Docker files or stack configuration changes.